LLD Phase 5 — PDF 3

LRU Cache · Rate Limiter · Digital Wallet · BookMyShow | Requirements first, hints in the middle, full solutions at the back

> **Colour key. Purple = the interviewer's words · Blue = a note or hint · Red = a common mistake · Green = a rule to remember · Amber = how to say it in an interview.**

Contents

Before you start — what's new in PDF 3 · the rush helper · time boxes · warm-up: two PDF 2 designs from memory, then 12 quick ones

Problem 1 — LRU Cache (small)

Problem 2 — Rate Limiter (core)

Problem 3 — Digital Wallet (core)

Problem 4 — BookMyShow (heavy)

Hints — three levels per problem: a nudge, the shape, the key decision

Solutions 1–4 — clarifying questions · Part A model answer · what to write first · full code · Part C answers · common mistakes · 10-point checklist

Back pages — warm-up answers · mistake log · re-do schedule

Before you start

The loop is the same as before: read once, Part A on paper, one hint level only after 15 minutes stuck, Part B until the output matches exactly, Part C out loud, then the solution and its checklist. What's new:

New in PDF 3	What it means for you
Questions as interviewers ask them	Each problem opens with the interviewer's words, in purple: short and a bit vague, the way it really sounds. The requirements after it are what good questions would get you. Before reading them, write down the questions *you* would ask.
Interview-sized solutions	The model code is what you could write in the round: about 100–200 lines of domain classes in plain Java (maps, enums, records, `synchronized`), only the checks the requirements ask for, few comments. Errors are thrown as `IllegalArgumentException` or `IllegalStateException`; a custom exception is nicer, but costs time.
"Write this first"	Each solution lists what to write first when time is short, and what's fine to explain out loud instead of coding.
The rush helper is given	Three problems test concurrency with a RUSH command. In an interview you write the thread-safe code, not the test harness, so the helper is below: paste it into your `Main`.

The rush helper (given)

```
// Given: runs task(0) .. task(n-1) on n threads that all start together; counts the trues.
static int rush(int n, IntPredicate task) throws Exception {
    ExecutorService pool = Executors.newFixedThreadPool(n);
    CountDownLatch go = new CountDownLatch(1);
    List<Future<Boolean>> results = new ArrayList<>();
    for (int i = 0; i < n; i++) {
        int id = i;
        results.add(pool.submit(() -> { go.await(); return task.test(id); }));
    }
    go.countDown();
    int trues = 0;
    for (Future<Boolean> f : results) if (f.get()) trues++;
    pool.shutdown();
    return trues;
}
```

It starts n threads, holds them at a "start gun" (the latch) so they all go at once, and counts how many tasks returned true. Use it as `rush(n, i -> ...)`, where `i` is the thread's number, from 0 to n − 1. It needs `import java.util.concurrent.*;` and `import java.util.function.IntPredicate;`, and `main` must declare `throws Exception`.

Problem	Size	Time box	Format
1. LRU Cache	small	45 min	10 design + 35 code
2. Rate Limiter	core	75 min	20 design + 55 code
3. Digital Wallet	core	75 min	20 design + 55 code
4. BookMyShow	heavy	100 min	25 design + 75 code

Warm-up (15 minutes)

Part 1 — from memory. Don't open PDF 2. Write on paper, then check against the back pages.

W1	**ATM** (5 min). Draw the state table: three states × six operations (insert, PIN, balance, withdraw, eject, load). Write the steps of a withdrawal in order and mark the commit point. Where does the wrong-PIN count live, and why there?
W2	**Elevator** (5 min). Which two decisions must be kept apart, and where does each one live? Which steps of a hall call are one indivisible step? What does a lift announce, and who listens? Why is one lock for the whole controller acceptable?

Part 2 — name the pattern (5 min). All from one app: an EV charging network across Pune. Name the pattern, or say "nothing" and what you'd use instead.

1. The price per kWh depends on the time of day (peak, off-peak, a flat night rate), and the business changes the rules every few months.

2. A charging session goes BOOKED → CHARGING → COMPLETED, or CANCELLED, or FAULTED. What the driver, the operator and the charger can each do depends on the status.

3. Chargers from one vendor are controlled through its library, `ocppSend(byte[] frame)`. Our code calls `Charger.start(sessionId)` everywhere.

4. When a session completes: send the bill, add loyalty points, free the slot in the app. More reactions are coming.

5. Refunds: up to Rs 200 are automatic, up to Rs 2,000 a support agent approves, above that the finance team.

6. Booking a slot: check the car's connector type, reserve the charger, take a deposit, send a QR code. The app, the website and partner apps all do this.

7. Stations have chargers, which have connectors. The dashboard wants the energy delivered at any level.

8. Users who haven't charged for 90 days get a "come back" coupon.

9. The map shows 2,000 chargers. The icon image is the same for every charger of a model, so it should be stored once.

10. The government subsidy API is slow. Check each user's eligibility once a day and reuse the answer, logging every real call.

11. A connector has a type (CCS2, Type 2) and a maximum power.

12. A trip plan has a start and an end (required) and many optional settings: avoid tolls, minimum battery on arrival, preferred networks. Once made, it doesn't change.

Problem 1 — LRU Cache

Small · 45 minutes (10 design + 35 code) · machine-coding format

> **The interviewer says:** "Design an in-memory cache with a fixed capacity. When it's full, it evicts the least recently used entry. `get` and `put` must both be O(1). Write it so we could plug in a different eviction policy later."

After your questions, the requirements are

R1	A cache has a capacity (at least 1) and stores key → value pairs. Keys and values are strings in the test, but the cache must work for any types (generics).
R2	`get(key)` returns the value, or reports a miss. A hit counts as a use of that key; a miss changes nothing.
R3	`put(key, value)` stores the value. If the key is new and the cache is full, first evict the least recently used key. Updating an existing key never evicts, and counts as a use.
R4	Both operations are O(1). `LinkedHashMap` is not allowed: the interviewer wants to see the data structure.
R5	Keep count of hits, misses and evictions.
NF1	The eviction policy can be replaced (LFU is planned) without changing the cache class.
NF2	Many request threads share one cache.

Out of scope: expiry by time (that's PDF 4's cache), sizes in bytes, saving to disk, a cache shared by many servers (Part C asks).

Commands and exact output

Command	Output
`CREATE capacity`	`Cache ready, capacity 3` (replaces any earlier cache; the input always starts with it)
`PUT key value`	`Put a`, or `Put d, evicted b`
`GET key`	`a = 1`, or `b: miss`
`STATS`	`Size 3 of 3, hits 3, misses 1, evictions 3`
`(error) capacity below 1`	`Capacity must be at least 1`

Part A — design

1	Which two data structures together give O(1) for "find this key" *and* "find the least recently used"? Draw them after `PUT a, PUT b, PUT c, GET a`.
2	Write `get` and `put` as pointer changes. What are a dummy head and a dummy tail for?
3	NF1: what methods does an eviction-policy interface need? Which class owns the values, and which owns the order of use?
4	`put` on an existing key when the cache is full: why must it not evict?

Part B — code

input.txt

```
CREATE 3
PUT a 1
PUT b 2
PUT c 3
GET a
PUT d 4
GET b
PUT c 30
PUT e 5
GET c
GET d
PUT f 6
STATS
CREATE 1
PUT x 1
PUT y 2
GET x
GET y
STATS
CREATE 0
```

expected output

```
Cache ready, capacity 3
Put a
Put b
Put c
a = 1
Put d, evicted b
b: miss
Put c
Put e, evicted a
c = 30
d = 4
Put f, evicted e
Size 3 of 3, hits 3, misses 1, evictions 3
Cache ready, capacity 1
Put x
Put y, evicted x
x: miss
y = 2
Size 1 of 1, hits 1, misses 1, evictions 1
Capacity must be at least 1
```

Part C — pressure questions

1	**Change.** "Switch to LFU: evict the least frequently used key; a tie goes to the least recently used." What changes, and how do you keep it O(1)?
2	**Concurrency.** "Why does `get` need the same lock as `put`? With 64 threads, that one lock is a bottleneck. What then?"
3	**Scale.** "We run ten app servers, each with its own cache, and users see stale data." What are your options?
4	**Production.** "Why didn't you just use `LinkedHashMap`?"
5	**Defend it.** "A separate eviction policy, for one policy? That's over-engineering."

Problem 2 — Rate Limiter

Core · 75 minutes (20 design + 55 code) · machine-coding format, with a concurrency test

> **The interviewer says:** "Design a rate limiter for our public API. Each client can make a limited number of requests per time window, depending on their plan. Implement fixed window, sliding window and token bucket, so we can compare them. Remember that many requests arrive at the same moment."

After your questions, the requirements are

R1	Clients register with a plan. **FREE** allows 3 requests per 12 seconds; **PREMIUM** allows 6 per 12 seconds.
R2	One algorithm is active for everyone. Choosing an algorithm resets every client's history. The default is FIXED_WINDOW.
R3	A request has a client and a time in whole seconds. One client's times never go backwards. Several requests can come in the same second; they're decided one after another.
R4	The algorithms, with L = the plan's limit and W = 12 seconds: **FIXED_WINDOW:** time is cut into windows [0, 12), [12, 24), ... A request is allowed if fewer than L requests were allowed in its window. **SLIDING_LOG:** a request at time t is allowed if fewer than L requests were allowed at times after t − W. Only allowed requests are remembered. **TOKEN_BUCKET:** the bucket holds up to L tokens and is full at the client's first request. Every W / L seconds after that first request, one token is added, never above L (FREE: every 4 s; PREMIUM: every 2 s). A request takes one token; no token, no request. A client's state starts with their first request after the algorithm was chosen.
NF1	A new algorithm must not change the service or the plans.
NF2	Many threads ask at the same moment. A client must never get more than their limit. (RUSH tests this.)

Out of scope: several servers (Part C asks), limits per endpoint, HTTP details such as a Retry-After header.

Commands and exact output

Command	Output
`CLIENT name plan`	`Client asha on FREE (3 per 12s)`
`ALGORITHM name`	`Algorithm: SLIDING_LOG`
`REQ client t n`	n requests at second t, decided one after another: `asha @10: 3 allowed, 0 rejected`
`RUSH client t n`	n threads each send one request at second t, at the same moment: `Rush ravi @40: 6 allowed, 94 rejected`
`(errors)`	`Unknown client bob` · `Unknown plan GOLD` · `Unknown algorithm LEAKY`

Part A — design

1	What varies, and what stays the same? Draw the classes.
2	For each algorithm: what does one client's state hold, and how much memory is that per client?
3	Where does thread-safety belong: in the service, in the map, or in each client's limiter? Why does `computeIfAbsent` matter here?
4	Find the fixed window's weakness in the expected output below.

Part B — code

input.txt

```
CLIENT asha FREE
CLIENT ravi PREMIUM
CLIENT kiran GOLD
REQ bob 0 1
ALGORITHM FIXED_WINDOW
REQ asha 10 3
REQ asha 11 1
REQ asha 12 3
ALGORITHM SLIDING_LOG
REQ asha 10 3
REQ asha 12 3
REQ asha 22 2
REQ asha 23 2
ALGORITHM TOKEN_BUCKET
REQ asha 10 4
REQ asha 13 1
REQ asha 14 1
REQ asha 30 5
REQ ravi 0 7
REQ ravi 5 3
RUSH ravi 40 100
ALGORITHM SLIDING_LOG
RUSH asha 50 100
ALGORITHM LEAKY
```

expected output

```
Client asha on FREE (3 per 12s)
Client ravi on PREMIUM (6 per 12s)
Unknown plan GOLD
Unknown client bob
Algorithm: FIXED_WINDOW
asha @10: 3 allowed, 0 rejected
asha @11: 0 allowed, 1 rejected
asha @12: 3 allowed, 0 rejected
Algorithm: SLIDING_LOG
asha @10: 3 allowed, 0 rejected
asha @12: 0 allowed, 3 rejected
asha @22: 2 allowed, 0 rejected
asha @23: 1 allowed, 1 rejected
Algorithm: TOKEN_BUCKET
asha @10: 3 allowed, 1 rejected
asha @13: 0 allowed, 1 rejected
asha @14: 1 allowed, 0 rejected
asha @30: 3 allowed, 2 rejected
ravi @0: 6 allowed, 1 rejected
ravi @5: 2 allowed, 1 rejected
Rush ravi @40: 6 allowed, 94 rejected
Algorithm: SLIDING_LOG
Rush asha @50: 3 allowed, 97 rejected
Unknown algorithm LEAKY
```

> **Check your token bucket by hand.** asha's first request is at t=10, so tokens come back at 14, 18, 22, ... At t=13 none has come back yet; at t=14 one has; by t=30 four would have, but the bucket holds only 3. If your numbers differ, look at what happens to the "last refill" time after a refill.

Part C — pressure questions

1	**Change.** "Add a sliding window counter: only two counters per client instead of a log." How does it decide?
2	**Scale.** "We now run five API servers behind a load balancer." What breaks, and what do you do?
3	**Failure.** "The shared store holding the counters is down. Do we allow everything, or block everything?"
4	**Memory.** "We have ten million clients."
5	**Defend it.** "Pick the best algorithm and hard-code it. Why a Strategy?"

Problem 3 — Digital Wallet

Core · 75 minutes (20 design + 55 code) · machine-coding format, with a concurrency test

> **The interviewer says:** "Build a digital wallet, like Paytm's. Users create a wallet with some money, send money to each other, and see their statement, and there's an overview of everyone's balance. We also run two offers. Offer 1: if after a transfer the sender and the receiver have the same balance, both get Rs 10. Offer 2: when we run it, the three users with the most transactions get Rs 10, 5 and 2. Make it easy to add more offers."

After your questions, the requirements are

R1	A wallet has a unique name and an opening balance in whole rupees.
R2	A transfer moves money from one wallet to another. Checked in this order: both wallets exist (the sender first), they're different, the amount is above zero, the sender has enough.
R3	Every transfer appears in both statements, in order: the sender sees `Ravi debit 25` and the receiver sees `Asha credit 25` (the other person, the type, the amount).
R4	**Offer 1:** right after a transfer, if the sender and the receiver have exactly the same balance, each gets Rs 10. Statement line: `Offer1 credit 10`.
R5	**Offer 2** (run on command): the 3 wallets with the most transfers, sent or received, get Rs 10, Rs 5 and Rs 2. Offer credits don't count as transfers. Ties: the higher balance first, then the wallet created first. Statement line: `Offer2 credit 10`.
R6	The overview lists every wallet with its balance, in the order they were created.
NF1	A new offer that reacts to transfers must not change the transfer code.
NF2	Transfers come from many threads. Money must never be lost or created, and two people sending to each other at the same moment must not freeze the system. (RUSH tests this.)

Out of scope: paise (Part C asks), withdrawals to a bank account, fixed deposits, KYC.

Commands and exact output

Command	Output
`CREATE name amount`	`Wallet created: Asha, Rs 100`
`TRANSFER from to amount`	`Transferred Rs 25 from Asha to Ravi` (offers show up in statements, not here)
`STATEMENT name`	one line per entry, oldest first; `No transactions for Neha` if there are none
`OVERVIEW`	one line per wallet: `Asha 85`
`OFFER2`	`Offer2: Asha +10, Ravi +5, Kiran +2`
`RUSH a b n amount`	n threads send a → b and n threads send b → a, all at the same moment, then: `Rush: 200 transfers, p1 300, p2 301`

Error	Output
`a name that's taken / an unknown wallet`	`Wallet Asha already exists` / `Unknown wallet Bob`
`sending to yourself / zero or less`	`Cannot transfer to yourself` / `Amount must be positive`
`not enough money`	`Insufficient balance in Meera's wallet`

Part A — design

1	The entities, and who owns what: the balance, the statement, the number of transfers.
2	NF1: Offer 1 and Offer 2 are different kinds of offer. What's the difference, and what interface does each one need?
3	NF2: what do you lock during a transfer? Two people send to each other at the same moment: what can go wrong, and what's the fix?
4	Write Offer 2's ranking as one `Comparator`.

Part B — code

input.txt

```
CREATE Asha 100
CREATE Ravi 50
CREATE Kiran 80
CREATE Meera 40
CREATE Asha 10
TRANSFER Asha Ravi 25
TRANSFER Kiran Meera 20
OVERVIEW
OFFER2
TRANSFER Meera Asha 15
TRANSFER Meera Kiran 5
TRANSFER Ravi Kiran 13
TRANSFER Asha Asha 5
TRANSFER Asha Bob 5
TRANSFER Meera Ravi 0
TRANSFER Meera Ravi 500
STATEMENT Asha
OVERVIEW
OFFER2
STATEMENT Meera
CREATE Neha 0
STATEMENT Neha
CREATE p1 300
CREATE p2 301
RUSH p1 p2 100 2
OVERVIEW
```

expected output

```
Wallet created: Asha, Rs 100
Wallet created: Ravi, Rs 50
Wallet created: Kiran, Rs 80
Wallet created: Meera, Rs 40
Wallet Asha already exists
Transferred Rs 25 from Asha to Ravi
Transferred Rs 20 from Kiran to Meera
Asha 85
Ravi 85
Kiran 70
Meera 70
Offer2: Asha +10, Ravi +5, Kiran +2
Transferred Rs 15 from Meera to Asha
Transferred Rs 5 from Meera to Kiran
Transferred Rs 13 from Ravi to Kiran
Cannot transfer to yourself
Unknown wallet Bob
Amount must be positive
Insufficient balance in Meera's wallet
Ravi debit 25
Offer1 credit 10
Offer2 credit 10
Meera credit 15
Asha 110
Ravi 77
Kiran 90
Meera 50
Offer2: Kiran +10, Meera +5, Asha +2
Kiran credit 20
Offer1 credit 10
Asha debit 15
Kiran debit 5
Offer2 credit 5
Wallet created: Neha, Rs 0
No transactions for Neha
Wallet created: p1, Rs 300
Wallet created: p2, Rs 301
Rush: 200 transfers, p1 300, p2 301
Asha 112
Ravi 77
Kiran 100
Meera 55
Neha 0
p1 300
p2 301
```

> **Why p1 300 and p2 301?** Their total is odd, so their balances can never be equal, and Offer 1 can never fire during the rush. Each has enough for all its sends, so every transfer succeeds in any order. The balances must end exactly where they started. If they don't, money was lost or created. If the program freezes, two threads are each waiting for the other's lock.

Part C — pressure questions

1	**Change.** "Add Offer 3: 1% cashback on every transfer above Rs 500." Where does it go, and does your interface allow it?
2	**Change.** "Amounts can now have paise." What changes?
3	**Failure.** "The server crashes in the middle of a transfer, after the debit and before the credit." In a real system, what prevents half a transfer?
4	**Scale.** "Wallets are now rows in a database, and we run several servers." How do you transfer safely?
5	**Defend it.** "Why lock in a fixed order? Just make `transfer()` `synchronized`."

Problem 4 — BookMyShow

Heavy · 100 minutes (25 design + 75 code) · machine-coding format, with a concurrency test

> **The interviewer says:** "Design BookMyShow. A user picks a city and a movie, sees the shows, chooses seats and pays. While they pay, the seats are held for a few minutes. Two users must never get the same seat."

After your questions, the requirements are

R1	A theatre is in a city and has screens. A screen has rows A, B, C, … with the same number of seats in each row (A1, A2, …). The last row is **PREMIUM** at Rs 300 a seat; the other rows are **REGULAR** at Rs 200.
R2	A show is one movie on one screen at one time (HH:MM). Each show has its own seat statuses: free, held or booked.
R3	Searching by city and movie lists the matching shows, sorted by time (then by show ID), with how many seats are free.
R4	**Hold:** a user asks for one or more seats of a show. All of them must exist and be free, or nothing is held. A seat asked for twice counts once. A successful hold creates a booking (BK1, BK2, …; failed attempts don't use up a number) in state **HELD**, for 5 minutes.
R5	Time is in minutes since the box office opened, and TIME moves the clock forward. When the clock reaches a held booking's expiry time, that booking is **RELEASED** and its seats are free again. Each release is announced.
R6	**Payment** works only on a HELD booking. Success makes it **CONFIRMED** and its seats booked. Failure releases it at once.
R7	**Cancel** works only on a CONFIRMED booking: its seats become free, the full amount is refunded, and it becomes **CANCELLED**.
NF1	Many users try to hold seats at the same moment. Never two holds on one seat (RUSH tests this). Users of different shows must not wait for each other.
NF2	The hold time and the prices will change; each should live in one place.

Out of scope: logins, the payment gateway itself, notifications (Part C asks), coupons, choosing seats for the user.

Commands and exact output

Command	Output
`THEATRE name city`	`Theatre Inox in Pune`
`SCREEN theatre id rows seats`	`Inox screen 1: 3x4`
`SHOW id theatre screen movie time`	`Show S1: Jawan, Inox 18:00`
`SHOWS city movie`	one line per show: `S2 PVR 15:30, 10 free`; or `No shows found`
`SEATS show`	one line per row: `A H H . .` (. free, H held, X booked)
`HOLD show user seat …`	`BK1 held: asha, S1 A1 A2, Rs 400, until 5`
`TIME minutes`	`Time 5`, then for each released booking, in booking order: `BK2 expired: C1 C2 released`
`PAY booking OK|FAIL`	`BK1 confirmed: asha, S1 A1 A2, Rs 400` / `BK3 payment failed: B1 B2 B3 released`
`CANCEL booking`	`BK1 cancelled, refund Rs 400`
`RUSH show n seat …`	n users try to hold the same seats at the same moment: `Rush: 1 held, 49 refused`

Error	Output
`unknown theatre / screen / show / booking`	`Unknown theatre Carnival` · `Unknown screen 9 at Inox` · `Unknown show S9` · `Unknown booking BK9`
`a seat that doesn't exist`	`Seat D1 does not exist`
`seats that aren't free`	`Seats not available: A2` (every such seat, in the order asked)
`paying a booking that isn't held / cancelling one that isn't confirmed`	`BK1 is not held` / `BK1 is not confirmed`

Which error wins? For HOLD: the show, then every seat exists, then every seat is free.

Part A — design

1	Entities and multiplicities: city, theatre, screen, seat, show, booking. Which seat facts belong to the screen, and which to the show?
2	Draw the booking's states and the commands that move it between them. Is this a job for the State pattern?
3	NF1: what exactly must be one indivisible step in a hold, and what do you lock? Why not one lock for the whole app?
4	Who releases a hold whose time is up, and when?

Part B — code

input.txt

```
THEATRE Inox Pune
THEATRE PVR Pune
THEATRE Cinepolis Mumbai
SCREEN Inox 1 3 4
SCREEN PVR 2 2 5
SCREEN Cinepolis 1 2 4
SCREEN Carnival 1 2 2
SHOW S1 Inox 1 Jawan 18:00
SHOW S2 PVR 2 Jawan 15:30
SHOW S3 Cinepolis 1 Jawan 18:00
SHOW S4 Inox 1 Dunki 21:00
SHOW S5 Inox 9 Jawan 21:00
SHOWS Pune Jawan
SHOWS Pune Pathaan
HOLD S1 asha A1 A2
HOLD S1 ravi A2 A3
HOLD S1 ravi C1 C2
HOLD S1 ravi D1
SEATS S1
TIME 3
PAY BK1 OK
PAY BK1 OK
TIME 5
PAY BK2 OK
HOLD S1 kiran B1 B2 B3
PAY BK3 FAIL
SEATS S1
RUSH S1 50 C3 C4
SEATS S1
CANCEL BK1
CANCEL BK1
SHOWS Pune Jawan
TIME 20
```

expected output

```
Theatre Inox in Pune
Theatre PVR in Pune
Theatre Cinepolis in Mumbai
Inox screen 1: 3x4
PVR screen 2: 2x5
Cinepolis screen 1: 2x4
Unknown theatre Carnival
Show S1: Jawan, Inox 18:00
Show S2: Jawan, PVR 15:30
Show S3: Jawan, Cinepolis 18:00
Show S4: Dunki, Inox 21:00
Unknown screen 9 at Inox
S2 PVR 15:30, 10 free
S1 Inox 18:00, 12 free
No shows found
BK1 held: asha, S1 A1 A2, Rs 400, until 5
Seats not available: A2
BK2 held: ravi, S1 C1 C2, Rs 600, until 5
Seat D1 does not exist
A H H . .
B . . . .
C H H . .
Time 3
BK1 confirmed: asha, S1 A1 A2, Rs 400
BK1 is not held
Time 5
BK2 expired: C1 C2 released
BK2 is not held
BK3 held: kiran, S1 B1 B2 B3, Rs 600, until 10
BK3 payment failed: B1 B2 B3 released
A X X . .
B . . . .
C . . . .
Rush: 1 held, 49 refused
A X X . .
B . . . .
C . . H H
BK1 cancelled, refund Rs 400
BK1 is not confirmed
S2 PVR 15:30, 10 free
S1 Inox 18:00, 10 free
Time 20
BK4 expired: C3 C4 released
```

Part C — pressure questions

1	**Change.** "Prices now depend on the show: weekend evening shows cost 20% more." Where does pricing go?
2	**Scale.** "Seats are rows in a database, and we run many servers." How do you make a hold safe?
3	**Failure.** "The bank took the money, but our server crashed before confirming the booking."
4	**Change.** "Send an SMS and an email when a booking is confirmed."
5	**Defend it.** "Why hold seats at all? Check them when the user pays. Holds waste seats."

Hints

Read one level at a time, and only after 15 minutes of being stuck. Each level gives away more than the last.

Problem 1 — LRU Cache

Hint 1 (a nudge). You need two things: find a key fast, and know the order in which keys were used. No single standard structure does both in O(1).

Hint 2 (the shape). A `HashMap` from key to node, plus a doubly linked list of nodes with the most recent at the front. Dummy head and tail nodes remove every null check. The policy owns the list and its map of nodes; the cache owns the values and the capacity.

Hint 3 (the key decision). Two helpers do all the work: `unlink(node)` and `addAfterHead(node)`. `get`: if present, tell the policy the key was used (move it to the front). `put`: if the key is new and the cache is full, ask the policy for a victim (the node just before the tail) and remove its value; then store, and tell the policy the key was used.

Problem 2 — Rate Limiter

Hint 1. Three algorithms behind one interface, and one limiter *object* per client, holding that client's state.

Hint 2. `RateLimiter.allow(now)`, with `FixedWindow` (window start, count), `SlidingLog` (a deque of times) and `TokenBucket` (tokens, last refill time). A service maps each client to a plan and to its limiter. A map from the algorithm's name to a small "maker" builds new limiters.

Hint 3. Each `allow()` is a check-then-act, so make it `synchronized`: the lock is per client. Create limiters with `computeIfAbsent`, so two racing threads can't make two. For the token bucket, use whole numbers: added = (now − lastRefill) / refillEvery, then move lastRefill forward by added × refillEvery (not to now).

Problem 3 — Digital Wallet

Hint 1. Offer 1 reacts to each transfer as it happens. Offer 2 runs on demand over all the wallets. They're different shapes, so they get different interfaces.

Hint 2. `Wallet` (name, balance, statement, creation number) · an `Entry` record for a statement line · `WalletService` with `transfer` · a `TransferOffer` interface with `EqualBalanceOffer` · a `TopTransactorsOffer` class with `run(wallets)`.

Hint 3. In `transfer`, lock both wallets, always the older one first (by creation number). Inside both locks: check the balance, debit, credit, then run the transfer offers. Offer 2's order: transfer count, highest first; then balance, highest first; then creation number. Count transfers from the statement instead of keeping a separate counter.

Problem 4 — BookMyShow

Hint 1. The screen knows the layout, which never changes. The show knows which seats are free for that show. One booking object moves from HELD to CONFIRMED, RELEASED or CANCELLED.

Hint 2. `Theatre` (city, screens) · `Screen` (rows, seats per row, seat type by row) · `Show` (movie, time, screen, a map of seat statuses) · `Booking` (user, seats, amount, expiry, status) · `BookingService` as the front door. Enums for the seat type, the seat status and the booking status.

Hint 3. `hold()`: inside `synchronized (show)`, check that every seat exists and is free, then mark them all held, then create the booking. TIME is the scheduler's tick: it releases held bookings whose expiry has come. Pay and cancel share one small method: "if the status is X, move it to Y and set the seats to Z", under the show's lock.

Solution 1 — LRU Cache

Questions a strong candidate asks first

Question	Answer assumed here
O(1) for both operations, or just "fast"?	O(1) for both
What goes when it's full? What about updating a key?	The least recently used; an update never evicts
Does a `get` count as a use? A miss?	A hit does; a miss changes nothing
What types are keys and values?	Any: generics
May I use `LinkedHashMap`?	No: build the list yourself
Is it used from many threads?	Yes

Part A — model answer

```
map:  { a → node a,  b → node b,  c → node c }        (after PUT a, PUT b, PUT c, GET a)

head ⇄ [a] ⇄ [c] ⇄ [b] ⇄ tail
       most recent        least recent: evicted next
```

1. A HashMap plus a doubly linked list. The map finds a key's node in O(1). The list keeps the order of use: a node can be cut out of the middle in O(1) because it knows both neighbours, and the least recently used is always next to the tail. Neither structure can do both jobs alone.

2. Pointer changes. `get` of a key: find its node in the map, unlink it (2 pointer changes), add it after the head (4 changes). `put` of a new key when full: the victim is `tail.prev`; unlink it and remove it from both maps; then add the new node after the head. The **dummy head and tail** mean every real node always has a node on each side, so "is this the first or the last?" never needs a null check.

3. Split the job. `EvictionPolicy<K>` has two methods: `keyUsed(key)` and `evict()`. The **cache** owns the values and the capacity, so it decides *when* to evict. The **policy** owns the order, so it decides *who*. That's Strategy: LFU is a new class (Part C 1), and the cache doesn't change.

4. Updates never evict. Updating an existing key doesn't add an entry, so the size doesn't grow. Evicting would throw a good entry away for nothing. The check is `!data.containsKey(key) && data.size() == capacity`.

```
Cache<K, V> ———> «interface» EvictionPolicy<K>  ◁- - -  LruPolicy<K> 1 ◆——— * Node
Cache<K, V> ———> Map<K, V> data                          LruPolicy<K> ———> Map<K, Node>
```

Thread safety (NF2): `get` and `put` are both `synchronized`. `get` is not a read: it moves a node. **Deliberately not used:** a Singleton, a Factory, a custom exception.

In the interview: write this first. (1) The interface, two methods. (2) `LruPolicy`: the `Node`, `unlink`, `addAfterHead`, then `keyUsed` and `evict`: about 15 minutes. (3) `Cache.get` and `put`: 10 minutes. (4) The driver. (5) The counters last. If time runs out, say "and the stats are three counters" rather than leaving `put` half done. The domain code here is about 95 lines.

Part B — the code

```
import java.util.*;

interface EvictionPolicy<K> {
    void keyUsed(K key);       // called after every hit and every put
    K evict();                 // forgets the victim and returns it
}

class LruPolicy<K> implements EvictionPolicy<K> {
    private class Node {
        final K key;
        Node prev, next;
        Node(K key) { this.key = key; }
    }

    private final Map<K, Node> nodes = new HashMap<>();
    private final Node head = new Node(null);     // head.next = most recently used
    private final Node tail = new Node(null);     // tail.prev = least recently used

    LruPolicy() {
        head.next = tail;
        tail.prev = head;
    }

    public void keyUsed(K key) {
        Node node = nodes.get(key);
        if (node == null) {
            node = new Node(key);
            nodes.put(key, node);
        } else {
            unlink(node);
        }
        addAfterHead(node);
    }

    public K evict() {
        Node victim = tail.prev;
        unlink(victim);
```

```
        nodes.remove(victim.key);
        return victim.key;
    }

    private void unlink(Node n) {
        n.prev.next = n.next;
        n.next.prev = n.prev;
    }

    private void addAfterHead(Node n) {
        n.prev = head;
        n.next = head.next;
        head.next.prev = n;
        head.next = n;
    }
}

class Cache<K, V> {
    private final int capacity;
    private final Map<K, V> data = new HashMap<>();
    private final EvictionPolicy<K> policy;
    private int hits, misses, evictions;

    Cache(int capacity, EvictionPolicy<K> policy) {
        if (capacity < 1) throw new IllegalArgumentException("Capacity must be at least 1");
        this.capacity = capacity;
        this.policy = policy;
    }

    synchronized Optional<V> get(K key) {
        if (!data.containsKey(key)) {
            misses++;
            return Optional.empty();
        }
        hits++;
        policy.keyUsed(key);
        return Optional.of(data.get(key));
    }

    // Returns the key that was evicted to make room, if any.
    synchronized Optional<K> put(K key, V value) {
        K evicted = null;
        if (!data.containsKey(key) && data.size() == capacity) {
            evicted = policy.evict();
            data.remove(evicted);
            evictions++;
        }
        data.put(key, value);
        policy.keyUsed(key);
        return Optional.ofNullable(evicted);
    }

    synchronized String stats() {
        return "Size " + data.size() + " of " + capacity + ", hits " + hits
                + ", misses " + misses + ", evictions " + evictions;
    }
}

public class Main {
    public static void main(String[] args) {
        Cache<String, String> cache = null;
        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
            try {
                switch (p[0]) {
                    case "CREATE" -> {
                        cache = new Cache<>(Integer.parseInt(p[1]), new LruPolicy<>());
                        System.out.println("Cache ready, capacity " + p[1]);
                    }
                    case "PUT" -> {
                        Optional<String> evicted = cache.put(p[1], p[2]);
                        String extra = evicted.map(k -> ", evicted " + k).orElse("");
                        System.out.println("Put " + p[1] + extra);
                    }
                    case "GET" -> System.out.println(
                            cache.get(p[1]).map(v -> p[1] + " = " + v).orElse(p[1] + ": miss"));
                    case "STATS" -> System.out.println(cache.stats());
                    default -> System.out.println("Unknown command: " + p[0]);
                }
            } catch (IllegalArgumentException e) {
                System.out.println(e.getMessage());
            }
        }
    }
}
```

How to read it: `keyUsed` handles both cases: a new key gets a new node; a known key's node is unlinked first. Either way it ends up after the head. `evict` always takes `tail.prev`. The cache never touches a node: it only tells the policy what happened.

Part C — model answers

1. LFU, in O(1). A new class behind the same interface; the cache doesn't change, and that's NF1 paying off. Keep each key's count, and for each count a `LinkedHashSet` of keys in the order they reached it, plus the smallest count in use. This was run against the cache above:

```
class LfuPolicy<K> implements EvictionPolicy<K> {
    private final Map<K, Integer> freq = new HashMap<>();
    private final Map<Integer, LinkedHashSet<K>> byFreq = new HashMap<>();   // oldest use first
    private int minFreq = 0;

    public void keyUsed(K key) {
        int f = freq.getOrDefault(key, 0);
        if (f > 0) {
            byFreq.get(f).remove(key);
            if (f == minFreq && byFreq.get(f).isEmpty()) minFreq++;
        } else {
            minFreq = 1;                                  // a new key has the lowest count
        }
        freq.put(key, f + 1);
        byFreq.computeIfAbsent(f + 1, x -> new LinkedHashSet<>()).add(key);
    }

    public K evict() {
        K victim = byFreq.get(minFreq).iterator().next();  // least used, then least recent
        byFreq.get(minFreq).remove(victim);
        freq.remove(victim);
        return victim;
    }
}
```

2. The lock and 64 threads. `get` moves a node, so it's a write, and a read-write lock doesn't help. Options: **lock striping**, splitting the cache into N smaller caches by `hash(key) % N`, each with its own lock and a share of the capacity (eviction becomes "least recent in that part", which is usually fine); or **approximate LRU**, as Redis does, sampling a few keys and evicting the oldest of those; or, in production, a library such as Caffeine, which records uses in buffers and reorders the list later.

3. Ten servers, stale data. (a) One shared cache, such as Redis or Memcached: one copy of the data, at the cost of a network call. Keys are spread across cache machines by consistent hashing. (b) Keep the local caches, but on every write, publish an "invalidate key X" message to all servers. (c) A short expiry time, so stale data lives only seconds. Many systems mix them: a small local cache with a short expiry in front of a shared one.

4. LinkedHashMap. In production, yes. It's this, and the interviewer will be glad you know it:

```
class LruCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;

    LruCache(int capacity) {
        super(16, 0.75f, true);                  // true = keep entries in access order
        this.capacity = capacity;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;                // evict the least recently used
    }
}
```

In the round, they forbid it to see whether you can build the structure yourself. Say both.

5. "A separate policy is over-engineering." "You'd be right if LRU were the only policy forever; one class with the list inside would be simpler. But LFU is already planned, and the split costs one interface with two methods. Now the cache only knows 'tell the policy, ask it for a victim', so LFU is a new class, and the cache and its tests don't change."

Common mistakes

- Only a HashMap, scanning for the oldest entry on eviction: O(n).
- Forgetting to move the node on `get`, so only `put` updates the order.
- Evicting on a `put` of an existing key.
- Removing the victim from the list but not from the value map (or the other way round).
- A singly linked list: cutting a node out of the middle then needs its predecessor, which is O(n).
- Leaving `get` unsynchronized because "it's only a read".

Score yourself (1 point each)

☐	A HashMap plus a doubly linked list, both used in O(1)	☐	Dummy head and tail, so no null checks
☐	A hit moves the key to the front; a miss changes nothing	☐	A new key when full evicts the node next to the tail first
☐	Updating an existing key never evicts	☐	The policy is an interface, and the cache doesn't know it's LRU
☐	The cache is generic	☐	`get` and `put` share one lock
☐	Your output matches exactly	☐	Part C 1: you explained an O(1) LFU

Solution 2 — Rate Limiter

Questions a strong candidate asks first

Question	Answer assumed here
Limits per client, per plan, or per endpoint?	Per client, set by their plan
What happens to a rejected request? Queue it?	Rejected straight away (HTTP 429); no queue
One server or many?	One for now (Part C 2)
Which algorithm?	All three, switchable, to compare them
How precise is time?	Whole seconds

Part A — model answer

```
RateLimitService ———> Map<String, Plan>          Plan: enum FREE(3), PREMIUM(6); WINDOW = 12 s
RateLimitService ———> Map<String, RateLimiter>   one per client, made at its first request
RateLimitService ———> LimiterMaker               chosen by name from Map<String, LimiterMaker>
«interface» RateLimiter  ◁- - -  FixedWindow, SlidingLog, TokenBucket
```

1. What varies: the algorithm. It's a **Strategy**, and the name-to-maker map is a small **Factory**. What stays the same: plans, finding a client's limiter, the driver. Each limiter object holds one client's state, so clients never share counters.

Algorithm	One client's state	Memory	Behaviour
FIXED_WINDOW	window start, count	2 numbers	simple, but up to 2L requests can pass in a moment at a window edge
SLIDING_LOG	times of allowed requests	up to L numbers	exact; memory grows with the limit
TOKEN_BUCKET	tokens, last refill time	2 numbers	allows a burst of L, then a steady rate; the most widely used

3. Thread safety lives in each limiter. `allow()` is a check-then-act ("is there room? then take it"), so it's `synchronized` on that client's limiter: one client's threads queue, but different clients never block each other. The maps are `ConcurrentHashMap`s, and `computeIfAbsent` makes "find or create this client's limiter" one step. With a `get` and then a `put`, two racing threads could each create a limiter, and each would allow L requests.

4. The fixed window's weakness, in the output. asha gets 3 requests at t=10 and 3 more at t=12: six in two seconds, twice her limit, because the window boundary at 12 resets the count. With the same requests, the sliding log refuses the second three (it's in the output), and so would the token bucket: no token comes back before t=14.

Testing a race, honestly. With `synchronized` removed from `allow()`, the RUSH test let 7 requests through (or 4 for FREE) in **2 runs out of 40**, on the 2-core machine used to prepare this PDF. Passing runs prove little; the reasoning in answer 3 is what makes it right.

In the interview: write this first. (1) The interface and `TokenBucket`, the one interviewers ask about most. (2) The service with `computeIfAbsent`. (3) The driver. (4) `SlidingLog`, then `FixedWindow`: each is about 15 lines. (5) RUSH, using the given helper. The domain code here is about 115 lines.

Part B — the code

```
import java.util.*;
import java.util.concurrent.*;
import java.util.function.IntPredicate;

enum Plan {
    FREE(3), PREMIUM(6);

    static final int WINDOW = 12;                    // seconds, for every plan
    final int limit;
    Plan(int limit) { this.limit = limit; }

    static Plan parse(String name) {
        for (Plan plan : values()) if (plan.name().equals(name)) return plan;
        throw new IllegalArgumentException("Unknown plan " + name);
    }
}

interface RateLimiter {                              // one object per client
    boolean allow(long now);
}

interface LimiterMaker {                             // builds a client's limiter from its plan
    RateLimiter make(int limit, int window);
}

class FixedWindow implements RateLimiter {
    private final int limit, window;
    private long windowStart = -1;
    private int count;

    FixedWindow(int limit, int window) { this.limit = limit; this.window = window; }

    public synchronized boolean allow(long now) {
        long start = now - now % window;
        if (start != windowStart) {                  // a new window: start counting again
            windowStart = start;
            count = 0;
```

```
        }
        if (count == limit) return false;
        count++;
        return true;
    }
}

class SlidingLog implements RateLimiter {
    private final int limit, window;
    private final Deque<Long> times = new ArrayDeque<>();   // allowed requests, oldest first

    SlidingLog(int limit, int window) { this.limit = limit; this.window = window; }

    public synchronized boolean allow(long now) {
        while (!times.isEmpty() && times.peekFirst() <= now - window) times.pollFirst();
        if (times.size() == limit) return false;
        times.addLast(now);
        return true;
    }
}

class TokenBucket implements RateLimiter {
    private final int capacity, refillEvery;
    private int tokens;
    private long lastRefill = -1;

    TokenBucket(int capacity, int refillEvery) {
        this.capacity = capacity;
        this.refillEvery = refillEvery;
        this.tokens = capacity;                       // starts full
    }

    public synchronized boolean allow(long now) {
        if (lastRefill < 0) lastRefill = now;         // the clock starts at the first request
        long added = (now - lastRefill) / refillEvery;
        if (added > 0) {
            tokens = (int) Math.min(capacity, tokens + added);
            lastRefill += added * refillEvery;
        }
        if (tokens == 0) return false;
        tokens--;
        return true;
    }
}

class RateLimitService {
    private static final Map<String, LimiterMaker> ALGORITHMS = Map.of(
            "FIXED_WINDOW", FixedWindow::new,
            "SLIDING_LOG", SlidingLog::new,
            "TOKEN_BUCKET", (limit, window) -> new TokenBucket(limit, window / limit));

    private final Map<String, Plan> plans = new ConcurrentHashMap<>();
    private final Map<String, RateLimiter> limiters = new ConcurrentHashMap<>();
    private volatile LimiterMaker algorithm = ALGORITHMS.get("FIXED_WINDOW");

    void register(String client, Plan plan) {
        plans.put(client, plan);
        limiters.remove(client);
    }

    void useAlgorithm(String name) {
        LimiterMaker chosen = ALGORITHMS.get(name);
        if (chosen == null) throw new IllegalArgumentException("Unknown algorithm " + name);
        algorithm = chosen;
        limiters.clear();                             // everyone starts again
    }

    boolean allow(String client, long now) {
        Plan plan = plans.get(client);
        if (plan == null) throw new IllegalArgumentException("Unknown client " + client);
        // computeIfAbsent: exactly one limiter per client, even when threads race
        RateLimiter limiter = limiters.computeIfAbsent(client,
                c -> algorithm.make(plan.limit, Plan.WINDOW));
        return limiter.allow(now);
    }
}

public class Main {
    public static void main(String[] args) throws Exception {
        RateLimitService service = new RateLimitService();
        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
            try {
                switch (p[0]) {
                    case "CLIENT" -> {
                        Plan plan = Plan.parse(p[2]);
                        service.register(p[1], plan);
                        System.out.println("Client " + p[1] + " on " + plan + " (" + plan.limit
                                + " per " + Plan.WINDOW + "s)");
                    }
                    case "ALGORITHM" -> {
                        service.useAlgorithm(p[1]);
                        System.out.println("Algorithm: " + p[1]);
                    }
                    case "REQ" -> {
                        long t = Long.parseLong(p[2]);
                        int n = Integer.parseInt(p[3]), allowed = 0;
                        for (int i = 0; i < n; i++) if (service.allow(p[1], t)) allowed++;
```

```
                        System.out.println(p[1] + " @" + t + ": " + allowed + " allowed, "
                                + (n - allowed) + " rejected");
                    }
                    case "RUSH" -> {
                        long t = Long.parseLong(p[2]);
                        int n = Integer.parseInt(p[3]);
                        int allowed = rush(n, i -> service.allow(p[1], t));
                        System.out.println("Rush " + p[1] + " @" + t + ": " + allowed + " allowed, "
                                + (n - allowed) + " rejected");
                    }
                    default -> System.out.println("Unknown command: " + p[0]);
                }
            } catch (IllegalArgumentException e) {
                System.out.println(e.getMessage());
            }
        }
    }

    // Given: runs task(0) .. task(n-1) on n threads that all start together; counts the trues.
    static int rush(int n, IntPredicate task) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(n);
        CountDownLatch go = new CountDownLatch(1);
        List<Future<Boolean>> results = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            int id = i;
            results.add(pool.submit(() -> { go.await(); return task.test(id); }));
        }
        go.countDown();
        int trues = 0;
        for (Future<Boolean> f : results) if (f.get()) trues++;
        pool.shutdown();
        return trues;
    }
}
```

How to read it: the token bucket refills lazily, when a request arrives, using whole numbers only, and moves `lastRefill` forward by whole intervals so no partial progress is lost. The maker for TOKEN_BUCKET turns a plan's limit and window into a refill interval (12 / 3 = 4 seconds). Choosing an algorithm clears the map, so every client starts again.

Part C — model answers

1. Sliding window counter. Keep two counts per client: the current fixed window and the previous one. Estimate the last W seconds as previous × (the part of the previous window still inside the last W seconds) + current. Example: W = 12, a request at t = 15 is 3 seconds into the window [12, 24); previous = 3, current = 1; estimate = 3 × 9/12 + 1 = 3.25, which isn't below 3, so reject. Two numbers per client, no edge burst, slightly approximate. It's one new class (NF1).

2. Five servers. Each server's memory sees only part of a client's traffic, so a client could get up to five times the limit. Options: (a) **a shared store** such as Redis, with an atomic step: `INCR` plus an expiry for a fixed window, a sorted set of times for a sliding log, or a small Lua script for a token bucket, so read-check-write happens as one step on the Redis server; (b) **sticky routing**, where the load balancer always sends a client to the same server: simple, but uneven, and a restart forgets the counts; (c) give each server a fifth of the limit: approximate.

3. The shared store is down. **Fail open** (allow everything) keeps the product working but drops the protection. **Fail closed** (reject everything) protects the back end but is an outage. The usual answer: fail open with a local per-server limiter as a fallback, and raise an alert. For a login endpoint that guards against password guessing, failing closed may be right. Say that it depends on what the limit protects.

4. Ten million clients. With Java objects, a sliding log costs a few hundred bytes per active client, which is gigabytes for ten million; a token bucket is two numbers. Two fixes: forget clients who've been quiet for longer than W (their state would reset anyway), using an LRU or expiring cache from Problem 1; and prefer a token bucket or a sliding window counter.

5. "Hard-code the best one." "You'd be right once we've decided: then I'd delete the other two. But the requirement is to compare them, and in practice different endpoints want different ones: login wants a strict sliding log, search can take bursts with a token bucket. Behind the interface the service doesn't care, and a sliding window counter is one more class."

Common mistakes

- One counter shared by every client.
- "Is there room?" and "take it" in two unlocked steps.
- `get`, then `put` if missing, to create limiters: two limiters in a race.
- A token bucket with `double` tokens and rounding drift, or setting `lastRefill = now` on every request, which throws away partial progress.
- A sliding log that also remembers rejected requests: a client who keeps trying can then never get through.
- Fixed window boundaries off by one: the window start is `now - now % W`.

Score yourself (1 point each)

☐	One `RateLimiter` interface, three classes, chosen by name	☐	One limiter object per client, created with `computeIfAbsent`
☐	Each `allow()` is `synchronized` (a lock per client)	☐	The fixed window resets at window boundaries
☐	The sliding log drops times at or before t − W and stores only allowed requests	☐	The token bucket starts full and refills with whole numbers
☐	Choosing an algorithm resets everyone	☐	Your output matches exactly, RUSH included
☐	You can point to the boundary burst in the output	☐	Part C 2: you named a shared store with an atomic step

Solution 3 — Digital Wallet

Questions a strong candidate asks first

Question	Answer assumed here
Whole rupees, or paise too?	Whole rupees (Part C 2 asks about paise)
Is there a minimum transfer?	Anything above zero
Do offer credits count as transactions for Offer 2?	No: only transfers, sent or received
How are Offer 2 ties broken?	Higher balance, then the wallet created first
Can Offer 1 fire again for the same two people?	Yes, after any transfer that makes them equal
Will transfers run at the same time?	Yes, from many threads

Part A — model answer

Entity	Kind	Owns
`Wallet`	class	its balance, its statement, its creation number; the transfer count is worked out from the statement
`Entry`	record	one statement line: the other party, credit or debit, the amount, whether it came from an offer
`WalletService`	class (front door)	creating wallets, transfers and their locks, running the offers
`TransferOffer` + `EqualBalanceOffer`	interface + class	Offer 1: reacts to each transfer (Observer)
`TopTransactorsOffer`	class	Offer 2: ranks all wallets when asked

```
WalletService 1 ◆——— * Wallet 1 ◆——— * Entry
WalletService ———> * «interface» TransferOffer  ◁- - -  EqualBalanceOffer      (after every transfer)
WalletService ———> TopTransactorsOffer                                          (when OFFER2 runs)
```

2. Two kinds of offer, two shapes. Offer 1 reacts to an event, so it's a listener: `afterTransfer(from, to)`. Offer 2 is a campaign over everyone, so it's `run(wallets)`. One interface for both would give each class a method it doesn't need. A new transfer-reacting offer is one more class in the list (NF1). Offer 1 runs **inside** the locks, because it must see both balances exactly as this transfer left them; it's part of this transfer's result, not a separate announcement.

3. Lock both wallets, always in the same order. A transfer changes two balances, so both are locked. Without a fixed order, Asha → Ravi locks Asha and waits for Ravi, while Ravi → Asha locks Ravi and waits for Asha: a **deadlock**, and both wait forever. The fix is one rule: always lock the older wallet (lower creation number) first. The balance check goes inside the locks, so nothing can change between checking and debiting. **Tested:** with the locks taken as "sender, then receiver", the RUSH test **froze in 10 runs out of 30**; with no locks at all, money was lost or created (p2 ended at 299 or 303) in **3 runs out of 100**. The rule makes it safe; the test only sometimes shows the danger.

4. Offer 2's ranking, as one Comparator.
`Comparator.comparingLong(Wallet::transferCount).reversed().thenComparing(Comparator.comparingLong(Wallet::balance).reversed()).thenComparingInt(w -> w.createdOrder)`. Note that the first `reversed()` reverses only what's built so far, the transfer count.

In the interview: write this first. (1) `Wallet`, `Entry`, `create`, and `transfer` with the lock order: about 25 minutes. (2) Statement and overview. (3) Offer 1 through the interface. (4) Offer 2's comparator. (5) RUSH, with the given helper. The domain code here is about 110 lines.

Part B — the code

```
import java.util.*;
import java.util.concurrent.*;
import java.util.function.IntPredicate;

// One statement line, e.g. "Ravi debit 50" or "Offer1 credit 10".
record Entry(String party, String type, long amount, boolean fromOffer) {
    @Override public String toString() { return party + " " + type + " " + amount; }
}

class Wallet {
    final String name;
    final int createdOrder;
    private long balance;
    private final List<Entry> statement = new ArrayList<>();

    Wallet(String name, long balance, int createdOrder) {
        this.name = name;
        this.balance = balance;
        this.createdOrder = createdOrder;
    }

    long balance() { return balance; }

    void debit(String party, long amount) {
        balance -= amount;
        statement.add(new Entry(party, "debit", amount, false));
    }

    void credit(String party, long amount, boolean fromOffer) {
        balance += amount;
        statement.add(new Entry(party, "credit", amount, fromOffer));
```

```
    }

    long transferCount() { return statement.stream().filter(e -> !e.fromOffer()).count(); }

    List<Entry> statement() { return List.copyOf(statement); }
}

interface TransferOffer {                // reacts to every transfer, while both wallets are locked
    void afterTransfer(Wallet from, Wallet to);
}

class EqualBalanceOffer implements TransferOffer {                        // Offer 1
    public void afterTransfer(Wallet from, Wallet to) {
        if (from.balance() == to.balance()) {
            from.credit("Offer1", 10, true);
            to.credit("Offer1", 10, true);
        }
    }
}

class TopTransactorsOffer {                                                // Offer 2, run on demand
    private static final long[] REWARDS = {10, 5, 2};

    List<String> run(Collection<Wallet> wallets) {
        List<Wallet> ranked = new ArrayList<>(wallets);
        ranked.sort(Comparator.comparingLong(Wallet::transferCount).reversed()
                .thenComparing(Comparator.comparingLong(Wallet::balance).reversed())
                .thenComparingInt(w -> w.createdOrder));
        List<String> given = new ArrayList<>();
        for (int i = 0; i < Math.min(REWARDS.length, ranked.size()); i++) {
            ranked.get(i).credit("Offer2", REWARDS[i], true);
            given.add(ranked.get(i).name + " +" + REWARDS[i]);
        }
        return given;
    }
}

class WalletService {
    private final Map<String, Wallet> wallets = new LinkedHashMap<>();      // in creation order
    private final List<TransferOffer> transferOffers = List.of(new EqualBalanceOffer());
    private final TopTransactorsOffer offer2 = new TopTransactorsOffer();

    void create(String name, long amount) {
        if (wallets.containsKey(name))
            throw new IllegalArgumentException("Wallet " + name + " already exists");
        wallets.put(name, new Wallet(name, amount, wallets.size()));
    }

    void transfer(String fromName, String toName, long amount) {
        Wallet from = wallet(fromName), to = wallet(toName);
        if (from == to) throw new IllegalArgumentException("Cannot transfer to yourself");
        if (amount <= 0) throw new IllegalArgumentException("Amount must be positive");
        // Always lock the older wallet first, so two opposite transfers can't deadlock.
        Wallet first = from.createdOrder < to.createdOrder ? from : to;
        Wallet second = (first == from) ? to : from;
        synchronized (first) {
            synchronized (second) {
                if (from.balance() < amount)
                    throw new IllegalStateException("Insufficient balance in " + from.name
                            + "'s wallet");
                from.debit(to.name, amount);
                to.credit(from.name, amount, false);
                for (TransferOffer offer : transferOffers) offer.afterTransfer(from, to);
            }
        }
    }

    List<String> runOffer2()  { return offer2.run(wallets.values()); }
    Collection<Wallet> all()  { return wallets.values(); }

    Wallet wallet(String name) {
        Wallet w = wallets.get(name);
        if (w == null) throw new IllegalArgumentException("Unknown wallet " + name);
        return w;
    }
}

public class Main {
    public static void main(String[] args) throws Exception {
        WalletService service = new WalletService();
        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
            try {
                switch (p[0]) {
                    case "CREATE" -> {
                        service.create(p[1], Long.parseLong(p[2]));
                        System.out.println("Wallet created: " + p[1] + ", Rs " + p[2]);
                    }
                    case "TRANSFER" -> {
                        service.transfer(p[1], p[2], Long.parseLong(p[3]));
                        System.out.println("Transferred Rs " + p[3] + " from " + p[1]
                                + " to " + p[2]);
                    }
                    case "STATEMENT" -> {
                        List<Entry> entries = service.wallet(p[1]).statement();
                        if (entries.isEmpty()) System.out.println("No transactions for " + p[1]);
                        entries.forEach(System.out::println);
                    }
```

```
                    case "OVERVIEW" -> {
                        for (Wallet w : service.all())
                            System.out.println(w.name + " " + w.balance());
                    }
                    case "OFFER2" -> {
                        List<String> given = service.runOffer2();
                        System.out.println("Offer2: " + String.join(", ", given));
                    }
                    case "RUSH" -> {
                        int n = Integer.parseInt(p[3]);
                        long amount = Long.parseLong(p[4]);
                        rush(2 * n, i -> {                  // even i one way, odd i the other
                            if (i % 2 == 0) service.transfer(p[1], p[2], amount);
                            else service.transfer(p[2], p[1], amount);
                            return true;
                        });
                        long a = service.wallet(p[1]).balance(), b = service.wallet(p[2]).balance();
                        System.out.println("Rush: " + 2 * n + " transfers, " + p[1] + " " + a
                                + ", " + p[2] + " " + b);
                    }
                    default -> System.out.println("Unknown command: " + p[0]);
                }
            } catch (IllegalArgumentException | IllegalStateException e) {
                System.out.println(e.getMessage());
            }
        }
    }

    // Given: runs task(0) .. task(n-1) on n threads that all start together; counts the trues.
    static int rush(int n, IntPredicate task) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(n);
        CountDownLatch go = new CountDownLatch(1);
        List<Future<Boolean>> results = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            int id = i;
            results.add(pool.submit(() -> { go.await(); return task.test(id); }));
        }
        go.countDown();
        int trues = 0;
        for (Future<Boolean> f : results) if (f.get()) trues++;
        pool.shutdown();
        return trues;
    }
}
```

How to read it: `transfer` runs every cheap check before taking any lock, then the balance check and all the changes inside both locks. The transfer count is computed from the statement ("compute, don't store"), so it can't drift. The wallet map is a plain `LinkedHashMap`, which is safe here because wallets are created before the rush starts; if wallets could be created during transfers, it would need to be a concurrent map.

Part C — model answers

1. Offer 3: cashback over Rs 500. It reacts to a transfer, so it's a new `TransferOffer`. But it needs the **amount**, and `afterTransfer(from, to)` doesn't pass it, so the interface must change. The lesson: pass the whole event, a `Transfer(from, to, amount)` record, so that future offers have everything they need. Then Offer 3 is `if (t.amount() > 500) t.from().credit("Offer3", t.amount() / 100, true)`, and no transfer code changes.

2. Paise. Store amounts as whole paise in a `long`, as in Splitwise. Parse and print at the edges (`BigDecimal` to read "12.50"), and write offer rewards as 1000 paise. Comparisons, locks and offers stay the same. Never `double`.

3. A crash between the debit and the credit. In this program both changes live in memory, so a crash loses both together. In a real system: one **database transaction** covering the debit, the credit and both statement rows, so it all happens or none of it does; a double-entry ledger, where every transfer is two rows that must balance; and an **idempotency key** on each request, so a retried request doesn't transfer twice.

4. Rows in a database, several servers. `synchronized` protects only one JVM. Inside one transaction, lock both rows with `SELECT … FOR UPDATE`, in wallet-ID order (the same deadlock rule; databases also detect deadlocks and abort one side). Or make the debit a conditional update, `UPDATE wallet SET balance = balance - 25 WHERE name = 'Asha' AND balance >= 25`, check that one row changed, then credit. Optimistic locking with a version column also works, with a retry.

5. "Just make transfer() synchronized." "You'd be right for a small app: one lock is simpler and can't deadlock, and I'd start there if traffic were low. But then every transfer waits for every other one, even between strangers. With a lock per wallet, Asha → Ravi and Kiran → Meera run at the same time, and the price is one rule: always lock the older wallet first."

Common mistakes
- Locking only the sender: the receiver's balance can be changed by another transfer at the same time.
- Locking in sender-then-receiver order: opposite transfers deadlock.
- Checking the balance before taking the locks.
- Running Offer 1 outside the locks, so it may see balances that a later transfer has already changed.
- Counting offer credits as transactions for Offer 2.

Score yourself (1 point each)

☐	Each wallet owns its balance and its statement	☐	Transfer checks come in the stated order
☐	Both wallets are locked, always the older one first	☐	The balance check is inside the locks
☐	Offer 1 is a `TransferOffer`, run inside the locks	☐	Offer 2's ranking is one Comparator with both tie-breaks
☐	Offer credits don't count as transactions	☐	Your output matches exactly, RUSH included
☐	Part C 1: you noticed the offer interface needs the amount	☐	Part C 3: you named a database transaction and an idempotency key

Solution 4 — BookMyShow

Questions a strong candidate asks first

Question	Answer assumed here
What can users search by?	City and movie
How long is a seat held while paying?	5 minutes
Seat types and prices?	Last row premium at Rs 300; the rest Rs 200
If one requested seat is taken, hold the others?	No: all or nothing
Cancellations and refunds?	Confirmed bookings, with a full refund
How many users at once?	Many; the same seats can be tried at the same moment

Part A — model answer

Entity	Kind	Owns
`Theatre`	record	name, city, screens
`Screen`	record	the layout: rows and seats per row; a seat's type by its row
`Show`	class	movie, time, screen, *this show's* seat statuses, and its own lock
`Booking`	class	user, show, seats, amount, expiry time, status
`SeatType` , `SeatStatus` , `BookingStatus`	enums	the price per type; free, held, booked; the booking's life
`BookingService`	class (front door)	setup, search, hold, the clock, pay, cancel

```
Theatre 1 ◆———— * Screen                        (the city is a field of Theatre)
Show ————> 1 Theatre,  Show ————> 1 Screen       Show 1 ◆———— * seat status (Map<String, SeatStatus>)
Booking ————> 1 Show,  Booking ————> * seat IDs
BookingService ————> * Theatre, * Show, * Booking

HOLD ——> HELD ——PAY OK——> CONFIRMED ——CANCEL——> CANCELLED
          ├——PAY FAIL——> RELEASED
          └——time up——> RELEASED
```

1. Layout on the screen, status on the show. "Is A1 a premium seat?" is a question for the screen, and the answer never changes. "Is A1 free for the 18:00 show?" is a question for the show, because the 21:00 show on the same screen has its own seats. Keeping statuses on the screen is the classic bug: booking A1 for one show would book it for every show.

2. An enum, not the State pattern. Only two operations care about the status: paying needs HELD, cancelling needs CONFIRMED. One small method, `change(id, from, to)` , checks the status and moves it. If many operations depended on the status, State would earn its place. Saying why you *didn't* use a pattern scores well.

3. One step, per show. In a hold: check that every seat exists and is free, mark them all held, create the booking. All of it inside `synchronized (show)` . Otherwise two users can both see A1 as free and both hold it. The lock is the **show**, not the whole app: one global lock would make every user in the country queue behind one popular show. While preparing this PDF, the show lock was removed as a test: RUSH then showed "2 held" in **5 runs out of 200**.

4. Expiry is a scheduled job. "Not paid within 5 minutes" isn't an event, so nothing would ever trigger it. A scheduler checks regularly and releases what's overdue. Here, TIME plays the scheduler's tick. In real code that's a `ScheduledExecutorService` running every few seconds, plus a check at payment time, in case the job hasn't run yet.

In the interview: write this first. (1) `Screen` , `Show` with its seat map, and `hold()` with the lock: about 25 minutes. This is the heart of the question. (2) `Booking` with `confirm` , `paymentFailed` and `cancel` through one `change()` : 15 minutes. (3) Expiry on TIME: 10 minutes. (4) Search and the seat map. (5) The setup commands and RUSH. Short on time? Describe the theatre and screen model on the board, and let each show carry its own layout. The domain code here is about 195 lines.

Part B — the code

```
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.IntPredicate;

enum SeatType {
    REGULAR(200), PREMIUM(300);
    final int price;
    SeatType(int price) { this.price = price; }
}

enum SeatStatus { FREE, HELD, BOOKED }

enum BookingStatus { HELD, CONFIRMED, RELEASED, CANCELLED }

record Screen(int id, int rows, int seatsPerRow) {
    String seatId(int row, int n) { return (char) ('A' + row) + String.valueOf(n); }

    SeatType typeOf(String seatId) {                          // the last row is premium
        return seatId.charAt(0) - 'A' == rows - 1 ? SeatType.PREMIUM : SeatType.REGULAR;
    }
```

```
}

record Theatre(String name, String city, Map<Integer, Screen> screens) {}

class Show {                                        // its seats are guarded by the show's lock
    final String id, movie, time;
    final Theatre theatre;
    final Screen screen;
    private final Map<String, SeatStatus> seats = new HashMap<>();

    Show(String id, String movie, String time, Theatre theatre, Screen screen) {
        this.id = id;
        this.movie = movie;
        this.time = time;
        this.theatre = theatre;
        this.screen = screen;
        for (int r = 0; r < screen.rows(); r++)
            for (int n = 1; n <= screen.seatsPerRow(); n++)
                seats.put(screen.seatId(r, n), SeatStatus.FREE);
    }

    synchronized SeatStatus status(String seat) { return seats.get(seat); }   // null: no such seat

    synchronized void set(List<String> ids, SeatStatus st) { ids.forEach(s -> seats.put(s, st)); }

    synchronized long freeCount() {
        return seats.values().stream().filter(st -> st == SeatStatus.FREE).count();
    }

    synchronized List<String> seatMap() {                    // one line per row: "A X X . ."
        List<String> lines = new ArrayList<>();
        for (int r = 0; r < screen.rows(); r++) {
            StringBuilder line = new StringBuilder().append((char) ('A' + r));
            for (int n = 1; n <= screen.seatsPerRow(); n++) {
                SeatStatus st = seats.get(screen.seatId(r, n));
                line.append(st == SeatStatus.FREE ? " ." : st == SeatStatus.HELD ? " H" : " X");
            }
            lines.add(line.toString());
        }
        return lines;
    }
}

class Booking {
    final int number;
    final String id, user;
    final Show show;
    final List<String> seats;
    final int amount, expiresAt;
    BookingStatus status = BookingStatus.HELD;               // changed only under the show's lock

    Booking(int number, String user, Show show, List<String> seats, int amount, int expiresAt) {
        this.number = number;
        this.id = "BK" + number;
        this.user = user;
        this.show = show;
        this.seats = seats;
        this.amount = amount;
        this.expiresAt = expiresAt;
    }

    String seatList() { return String.join(" ", seats); }
    String summary()  { return user + ", " + show.id + " " + seatList() + ", Rs " + amount; }
}

class BookingService {
    static final int HOLD_MINUTES = 5;
    private final Map<String, Theatre> theatres = new HashMap<>();
    private final Map<String, Show> shows = new HashMap<>();
    private final Map<String, Booking> bookings = new ConcurrentHashMap<>();
    private final AtomicInteger counter = new AtomicInteger();
    private volatile int now = 0;                            // minutes since the box office opened

    void addTheatre(String name, String city) {
        theatres.put(name, new Theatre(name, city, new HashMap<>()));
    }

    void addScreen(String theatre, int id, int rows, int seatsPerRow) {
        theatre(theatre).screens().put(id, new Screen(id, rows, seatsPerRow));
    }

    void addShow(String id, String theatreName, int screenId, String movie, String time) {
        Theatre t = theatre(theatreName);
        Screen screen = t.screens().get(screenId);
        if (screen == null)
            throw new IllegalArgumentException("Unknown screen " + screenId + " at " + t.name());
        shows.put(id, new Show(id, movie, time, t, screen));
    }

    List<Show> find(String city, String movie) {
        return shows.values().stream()
                .filter(s -> s.theatre.city().equals(city) && s.movie.equals(movie))
                .sorted(Comparator.comparing((Show s) -> s.time).thenComparing(s -> s.id))
                .toList();
    }

    Booking hold(String showId, String user, List<String> requested) {
        Show show = show(showId);
        List<String> seats = new ArrayList<>(new LinkedHashSet<>(requested));  // repeats count once
        synchronized (show) {                                // check and hold: one step per show
            for (String s : seats)
```

```
                if (show.status(s) == null)
                    throw new IllegalArgumentException("Seat " + s + " does not exist");
            List<String> taken = seats.stream()
                    .filter(s -> show.status(s) != SeatStatus.FREE).toList();
            if (!taken.isEmpty())
                throw new IllegalStateException("Seats not available: " + String.join(" ", taken));
            show.set(seats, SeatStatus.HELD);
            int amount = seats.stream().mapToInt(s -> show.screen.typeOf(s).price).sum();
            int number = counter.incrementAndGet();         // only successful holds take a number
            Booking b = new Booking(number, user, show, seats, amount, now + HOLD_MINUTES);
            bookings.put(b.id, b);
            return b;
        }
    }

    // Moving the clock is the scheduler's tick: every hold whose time is up is released.
    List<Booking> advanceClock(int minutes) {
        now = minutes;
        List<Booking> released = new ArrayList<>();
        List<Booking> due = bookings.values().stream().filter(b -> b.expiresAt <= minutes)
                .sorted(Comparator.comparingInt(b -> b.number)).toList();
        for (Booking b : due) {
            synchronized (b.show) {
                if (b.status == BookingStatus.HELD) {
                    move(b, BookingStatus.RELEASED, SeatStatus.FREE);
                    released.add(b);
                }
            }
        }
        return released;
    }

    Booking confirm(String id) {                            // payment went through
        return change(id, BookingStatus.HELD, BookingStatus.CONFIRMED);
    }

    Booking paymentFailed(String id) {
        return change(id, BookingStatus.HELD, BookingStatus.RELEASED);
    }

    Booking cancel(String id) {
        return change(id, BookingStatus.CONFIRMED, BookingStatus.CANCELLED);
    }

    private Booking change(String id, BookingStatus from, BookingStatus to) {
        Booking b = bookings.get(id);
        if (b == null) throw new IllegalArgumentException("Unknown booking " + id);
        synchronized (b.show) {
            if (b.status != from)
                throw new IllegalStateException(b.id + " is not " + from.name().toLowerCase());
            move(b, to, to == BookingStatus.CONFIRMED ? SeatStatus.BOOKED : SeatStatus.FREE);
            return b;
        }
    }

    private void move(Booking b, BookingStatus status, SeatStatus seatStatus) {
        b.show.set(b.seats, seatStatus);
        b.status = status;
    }

    Show show(String id) {
        Show s = shows.get(id);
        if (s == null) throw new IllegalArgumentException("Unknown show " + id);
        return s;
    }

    private Theatre theatre(String name) {
        Theatre t = theatres.get(name);
        if (t == null) throw new IllegalArgumentException("Unknown theatre " + name);
        return t;
    }
}

public class Main {
    public static void main(String[] args) throws Exception {
        BookingService service = new BookingService();
        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
            List<String> seats = Arrays.asList(p).subList(Math.min(3, p.length), p.length);
            try {
                switch (p[0]) {
                    case "THEATRE" -> {
                        service.addTheatre(p[1], p[2]);
                        System.out.println("Theatre " + p[1] + " in " + p[2]);
                    }
                    case "SCREEN" -> {
                        service.addScreen(p[1], Integer.parseInt(p[2]), Integer.parseInt(p[3]),
                                Integer.parseInt(p[4]));
                        System.out.println(p[1] + " screen " + p[2] + ": " + p[3] + "x" + p[4]);
                    }
                    case "SHOW" -> {
                        service.addShow(p[1], p[2], Integer.parseInt(p[3]), p[4], p[5]);
                        System.out.println("Show " + p[1] + ": " + p[4] + ", " + p[2] + " " + p[5]);
                    }
                    case "SHOWS" -> {
                        List<Show> found = service.find(p[1], p[2]);
                        if (found.isEmpty()) System.out.println("No shows found");
                        for (Show s : found)
```

```
                        System.out.println(s.id + " " + s.theatre.name() + " " + s.time + ", "
                                + s.freeCount() + " free");
                }
                case "SEATS" -> service.show(p[1]).seatMap().forEach(System.out::println);
                case "HOLD" -> {
                    Booking b = service.hold(p[1], p[2], seats);
                    System.out.println(b.id + " held: " + b.summary() + ", until "
                            + b.expiresAt);
                }
                case "TIME" -> {
                    List<Booking> released = service.advanceClock(Integer.parseInt(p[1]));
                    System.out.println("Time " + p[1]);
                    for (Booking b : released)
                        System.out.println(b.id + " expired: " + b.seatList() + " released");
                }
                case "PAY" -> {
                    if (p[2].equals("OK")) {
                        Booking b = service.confirm(p[1]);
                        System.out.println(b.id + " confirmed: " + b.summary());
                    } else {
                        Booking b = service.paymentFailed(p[1]);
                        System.out.println(b.id + " payment failed: " + b.seatList()
                                + " released");
                    }
                }
                case "CANCEL" -> {
                    Booking b = service.cancel(p[1]);
                    System.out.println(b.id + " cancelled, refund Rs " + b.amount);
                }
                case "RUSH" -> {                          // n users, same seats, same moment
                    int n = Integer.parseInt(p[2]);
                    int held = rush(n, i -> {
                        try {
                            service.hold(p[1], "user" + i, seats);
                            return true;
                        } catch (IllegalStateException e) {
                            return false;
                        }
                    });
                    System.out.println("Rush: " + held + " held, " + (n - held) + " refused");
                }
                default -> System.out.println("Unknown command: " + p[0]);
            }
        } catch (IllegalArgumentException | IllegalStateException e) {
            System.out.println(e.getMessage());
        }
    }
}

// Given: runs task(0) .. task(n-1) on n threads that all start together; counts the trues.
static int rush(int n, IntPredicate task) throws Exception {
    ExecutorService pool = Executors.newFixedThreadPool(n);
    CountDownLatch go = new CountDownLatch(1);
    List<Future<Boolean>> results = new ArrayList<>();
    for (int i = 0; i < n; i++) {
        int id = i;
        results.add(pool.submit(() -> { go.await(); return task.test(id); }));
    }
    go.countDown();
    int trues = 0;
    for (Future<Boolean> f : results) if (f.get()) trues++;
    pool.shutdown();
    return trues;
}
}
```

How to read it: `hold()` is the one indivisible step of answer 3. Every later change to a booking (confirm, fail, cancel, expire) also takes the show's lock, so a payment and an expiry can't both act on the same booking. Booking numbers come from an `AtomicInteger`, taken only after the seats are secured, so failed attempts don't use up numbers.

Part C — model answers

1. Prices that depend on the show. Pricing becomes a rule of its own: a `PricingStrategy` with `price(show, seat)`. The base price still comes from the seat type, and a weekend-evening strategy multiplies it by 1.2. The service asks the strategy instead of the enum. If several surcharges must stack (weekend, 3D, recliner), wrap them as decorators.

2. Rows in a database, many servers. `synchronized` protects one JVM only. Keep a `show_seat` table (show, seat, status, booking). A hold is one transaction: `UPDATE show_seat SET status = 'HELD', booking_id = ? WHERE show_id = ? AND seat_id IN (...) AND status = 'FREE'`. If fewer rows changed than seats asked for, roll back. Alternatives: a unique constraint on (show, seat) in a seat-lock table, so a second insert fails; or Redis `SET … NX` with a 5-minute expiry per seat.

3. Paid, but not confirmed. The hold will expire and the seats could be sold again, while the customer's money is taken. Fixes: confirm from the payment gateway's callback, made safe to repeat by keying it on the booking ID; a reconciliation job that matches payments with bookings; if the seats are already gone, refund automatically. Many apps also extend the hold while a payment is in progress.

4. SMS and email on confirmation. Observer: a `BookingListener` with `onConfirmed(booking)`, and `SmsNotifier` and `EmailNotifier` classes. Call them after the status change and outside the show's lock. A failed SMS must not undo the booking: retry it from a queue.

5. "Why hold seats at all?" "You'd be right that holds waste seats when people abandon payment, and for a small cinema with few clashes, checking at payment time could work. But then a user can type in their card details and lose the seat at the last second, which is the worst experience on a sold-out show. A short hold, released automatically, is the trade-off, and the hold time is one constant we can tune."

Common mistakes

- Checking the seats and holding them in separate steps, or holding them one at a time, so a failed hold leaves some seats held.
- One lock for every show.
- Seat statuses on the screen instead of the show.
- Never releasing expired holds, so seats are lost until restart.
- Letting an expired or failed booking be paid.
- A State class per booking status, for two operations.

Score yourself (1 point each)

☐	The layout is on the screen; seat statuses are on the show	☐	One `Booking` with a status: HELD, CONFIRMED, RELEASED, CANCELLED
☐	A hold checks every seat and marks them all in one locked step	☐	The lock is per show
☐	The clock releases held bookings when their time is up	☐	Paying needs HELD; cancelling needs CONFIRMED
☐	Prices come from one place (the seat type)	☐	Your output matches exactly, RUSH included
☐	Part C 2: you described a conditional update or a unique constraint	☐	Part C 3: you handled "paid but not confirmed"

Back pages

Warm-up answers

W1 — ATM. The state table:

	insert	PIN	balance	withdraw	eject	load
IDLE	check the card → CARD_INSERTED	"Insert a card first"	"Insert…"	"Insert…"	"Insert…"	allowed
CARD_INSERTED	"already inserted"	right → AUTHENTICATED; wrong → count; third → block, end session	"Enter your PIN first"	"Enter…"	→ IDLE	"during a session"
AUTHENTICATED	"already inserted"	"already accepted"	show it	plan, debit, take	→ IDLE	"during a session"

A withdrawal: (1) check the amount and the limit; (2) plan the notes on paper, changing nothing; (3) debit the account, checking and subtracting in one locked step: **the commit point**; (4) take the notes out of the cassettes. A failure after step 3 needs compensation (credit the money back). The wrong-PIN count lives **on the card**, at the bank: in the ATM or the session, ejecting and inserting again would reset it.

W2 — Elevator. "Which lift answers a hall call?" lives in the dispatcher, as a **Strategy** chosen by name; "in what order does a lift visit its stops?" lives in the lift's own states (keep going while there are stops ahead, then turn). One indivisible step: is the button lit + choose a lift + add the stop + light the button. A lift announces "I stopped at floor f"; the hall buttons (turn the light off) and the display listen, and the lift knows neither. One lock is fine because a building has a handful of lifts and each operation takes microseconds, and it rules out deadlock.

#	W3 answer	Why
1	**Strategy**	different pricing rules, changing often. (If only the rates differed, an enum with fields.)
2	**State**	several operations, each depending on the status.
3	**Adapter**	a vendor's interface we can't change, behind our own.
4	**Observer**	reactions to one event, with more to come.
5	**Chain of Responsibility**	up to X this one decides; otherwise pass it on.
6	**Facade**	several steps in order behind one call, used from three places.
7	**Composite**	a tree where a whole and its parts answer the same question.
8	**Nothing**: a scheduled job	"hasn't happened in N days" isn't an event. Run a daily job.
9	**Flyweight**	the same unchanging data shared by thousands of objects, to save memory.
10	**Proxy** (one that remembers)	the same interface as the real API, keeping answers for a day and logging real calls.
11	**Nothing**: a record	a couple of fields that never change, with an enum for the type.
12	**Builder**	required parts plus many optional ones, never changing afterwards.

Both designs right and 10 or more of the 12: on track. If W1 or W2 was shaky, re-do that problem's Part A from PDF 2 before Problem 2 here.

Mistake log

One row per mistake, in your own words. Before starting PDF 4, read this page and the logs in PDFs 1 and 2 once.

Date	Problem	What I did	What's right	Rule to remember

Re-do schedule

Problem	Attempted (time taken)	Day 3: Part A from memory (15 min)	Day 10: Part A + the core class	Day 30: the whole problem, timed	Score /10
LRU Cache					
Rate Limiter					
Digital Wallet					
BookMyShow					

The core class to rebuild on day 10: `LruPolicy` (the list) · `TokenBucket` and the service's `computeIfAbsent` · `WalletService.transfer` with the lock order · `BookingService.hold`. PDF 2's day-30 re-dos are probably due around now too.

Next: PDF 4. Snake and Ladder, a Cache with TTL and eviction, a Meeting Room Scheduler and Chess. Its warm-up will ask you to re-design the Rate Limiter and BookMyShow from memory, so the day-10 re-dos above are worth doing before you open it.